

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("/content/Indian_Startups_Dataset.csv", encoding='latin1')
```

```
df.head()
```

	Company_Name	Market	Country_Code	City	Founded_Year	Funding_Rounds	Funding_Total_USD	First_Funding_At	Last_
0	1CLICK	Chat	IND	Bengaluru	2012.0	1	0	2014-01-26	
1	1mg	E-Commerce	IND	Gurgaon	2015.0	16	204600209	07-05-2015	
2	21Diamonds	E-Commerce	IND	Gurgaon	2012.0	1	6369507	2012-11-15	
3	24x7 Learning	Education	IND	Bengaluru	2001.0	1	4000000	2007-11-06	
4	3DSoC	Mobile	IND	Bengaluru	2006.0	2	2065000	2007-12-01	

```
df.shape
```

```
(1035, 10)
```

```
df = df[df["Founded_Year"] >= 2000]
df.head()
```

	Company_Name	Market	Country_Code	City	Founded_Year	Funding_Rounds	Funding_Total_USD	First_Funding_At	Last_
0	1CLICK	Chat	IND	Bengaluru	2012.0	1	0	2014-01-26	
1	1mg	E-Commerce	IND	Gurgaon	2015.0	16	204600209	07-05-2015	
2	21Diamonds	E-Commerce	IND	Gurgaon	2012.0	1	6369507	2012-11-15	
3	24x7 Learning	Education	IND	Bengaluru	2001.0	1	4000000	2007-11-06	
4	3DSoC	Mobile	IND	Bengaluru	2006.0	2	2065000	2007-12-01	

```
df.shape
```

```
(806, 10)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 806 entries, 0 to 1034
Data columns (total 10 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Company_Name        806 non-null    object
1   Market              787 non-null    object
2   Country_Code        806 non-null    object
3   City                804 non-null    object
4   Founded_Year        806 non-null    float64
5   Funding_Rounds      806 non-null    int64
6   Funding_Total_USD   806 non-null    int64
7   First_Funding_At    806 non-null    object
8   Last_Funding_At     806 non-null    object
9   Status              792 non-null    object
dtypes: float64(1), int64(2), object(7)
memory usage: 69.3+ KB
```

```
df.isnull().sum()
```

	0
Company_Name	0
Market	19
Country_Code	0
City	2
Founded_Year	0
Funding_Rounds	0
Funding_Total_USD	0
First_Funding_At	0
Last_Funding_At	0
Status	14

dtype: int64

```
df.dropna(inplace=True)
```

```
df.shape
```

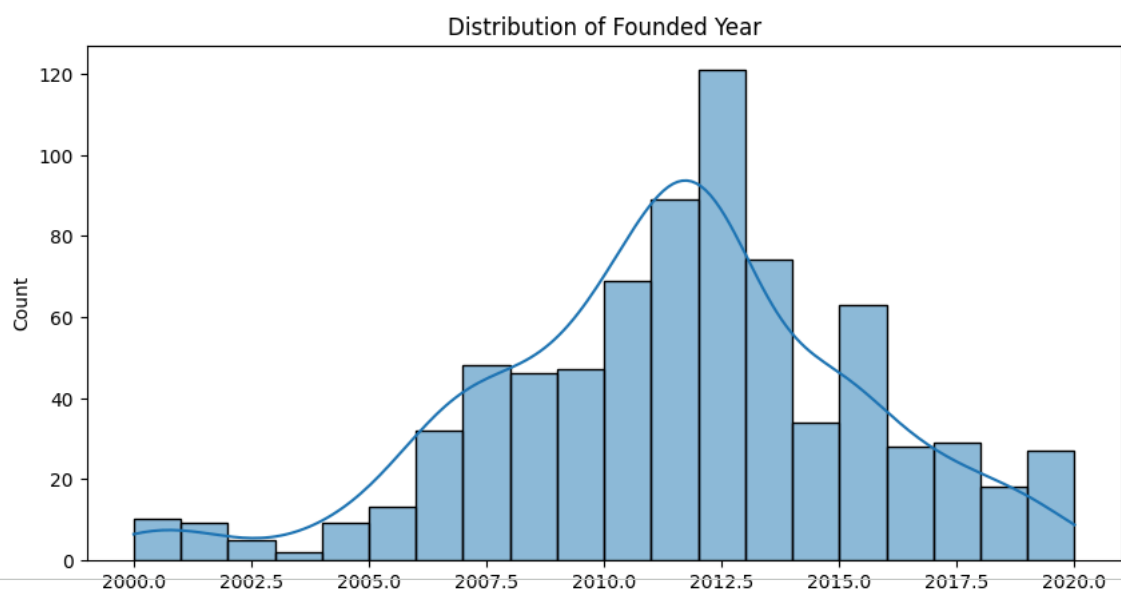
```
(773, 10)
```

```
df.drop_duplicates(inplace=True)
df.shape
```

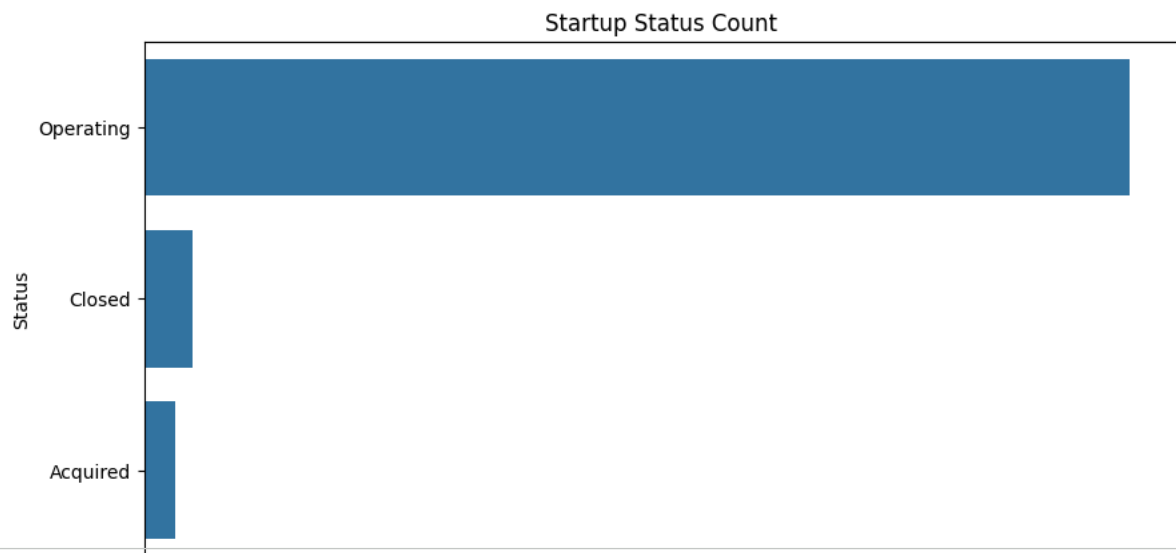
```
(773, 10)
```

```
# Create survival time (years from founding to last funding or 2026 for censored)
df['First_Funding_At'] = pd.to_datetime(df['First_Funding_At'], errors='coerce')
df['Last_Funding_At'] = pd.to_datetime(df['Last_Funding_At'], errors='coerce')
df['Founded_Year'] = df['Founded_Year'].fillna(2000).astype(int)
```

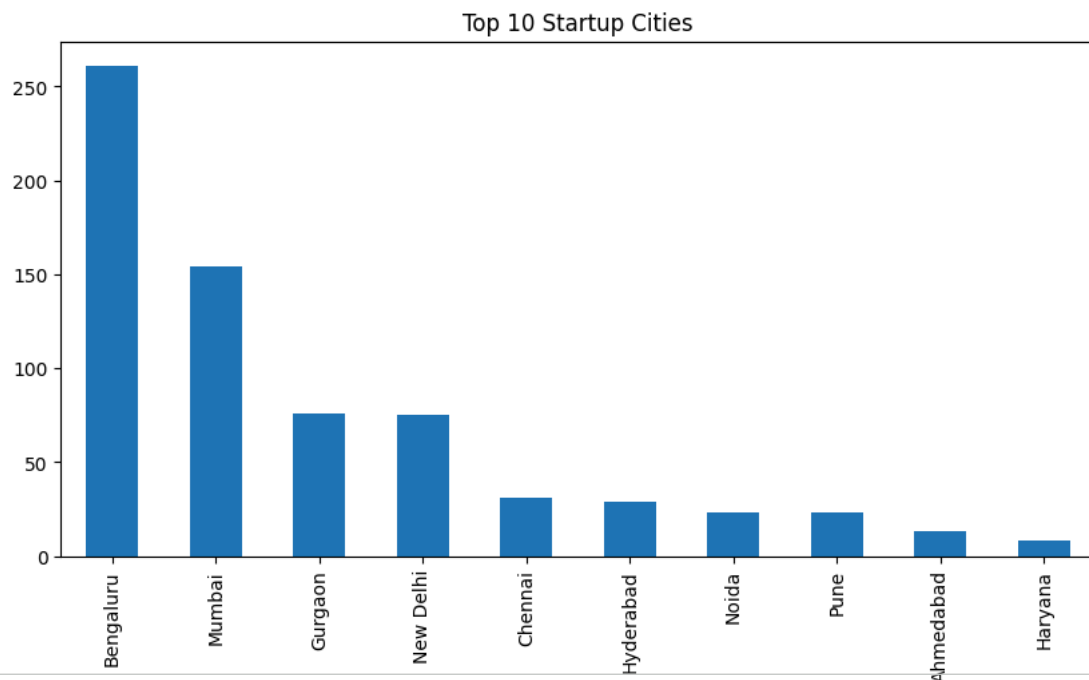
```
# Startup Founded Year Distribution
plt.figure(figsize=(10,5))
sns.histplot(df["Founded_Year"].dropna(), bins=20, kde=True)
plt.title("Distribution of Founded Year")
plt.xlabel("Founded Year")
plt.show()
```



```
# Startup Status Count
plt.figure(figsize=(10,5))
sns.countplot(y=df["Status"])
plt.title("Startup Status Count")
plt.show()
```



```
# Top 10 Cities
plt.figure(figsize=(10,5))
df["City"].value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Startup Cities")
plt.show()
```

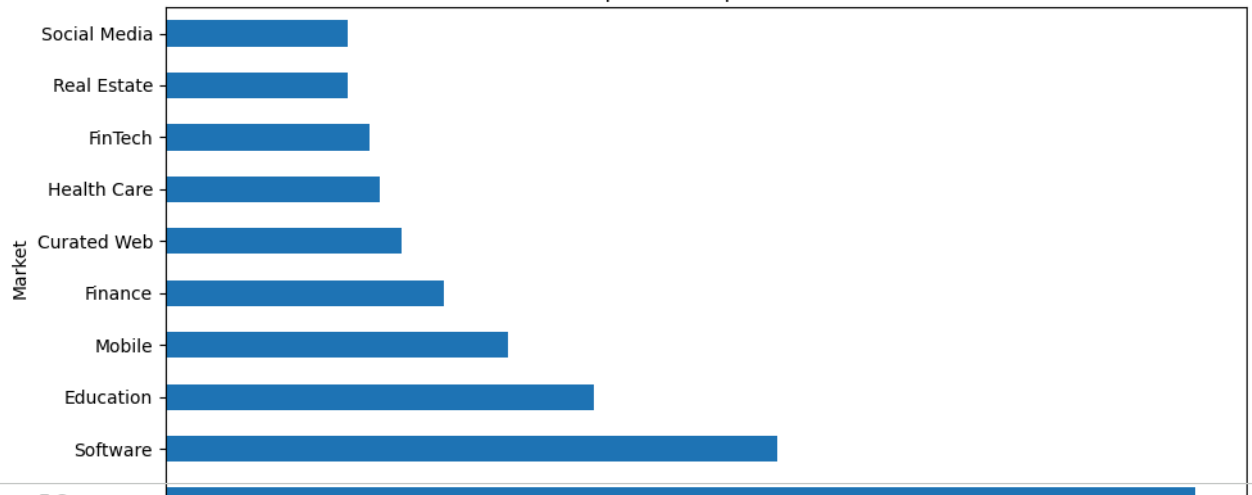


```
# Strip whitespace first, then replace
df["Market"] = df["Market"].str.strip()
df["Market"] = df["Market"].replace(["E-Commerce", "E - Commerce", "Ecommerce"], "E-Commerce")

# Now show top 10
top_markets = df["Market"].value_counts().head(10).sort_values(ascending=False)

# Plot horizontal bar chart
top_markets.plot(kind="barh", figsize=(10,5), title="Top 10 Startup Markets")
plt.xlabel("Count")
plt.tight_layout()
plt.show()
```

Top 10 Startup Markets



```
# Create event indicator: 1 if closed/acquired, 0 if operating
status_map = {
    'operating': 0, 'Operating': 0, 'Operating ': 0,
    'closed': 1, 'Closed': 1,
    'acquired': 1, 'Acquired': 1
}
df['Event'] = df['Status'].str.lower().map(status_map).fillna(0)

# Drop rows with missing critical data
df = df.dropna(subset=['Market', 'Funding_Rounds', 'Funding_Total_USD'])

print(f"Final dataset size: {len(df)} startups")
print(f"Event rate: {df['Event'].mean()*100:.1f}% (closed/acquired)")
```

Final dataset size: 773 startups
Event rate: 7.4% (closed/acquired)

```
current_year = 2026

def calc_survival_time(row):
    if row['Event'] == 1:
        if pd.notna(row['Last_Funding_At']):
            return row['Last_Funding_At'].year - row['Founded_Year']
        else:
            return current_year - row['Founded_Year']
    else:
        return current_year - row['Founded_Year']

df['Survival_Years'] = df.apply(calc_survival_time, axis=1)

df = df[df['Survival_Years'] > 0]
```

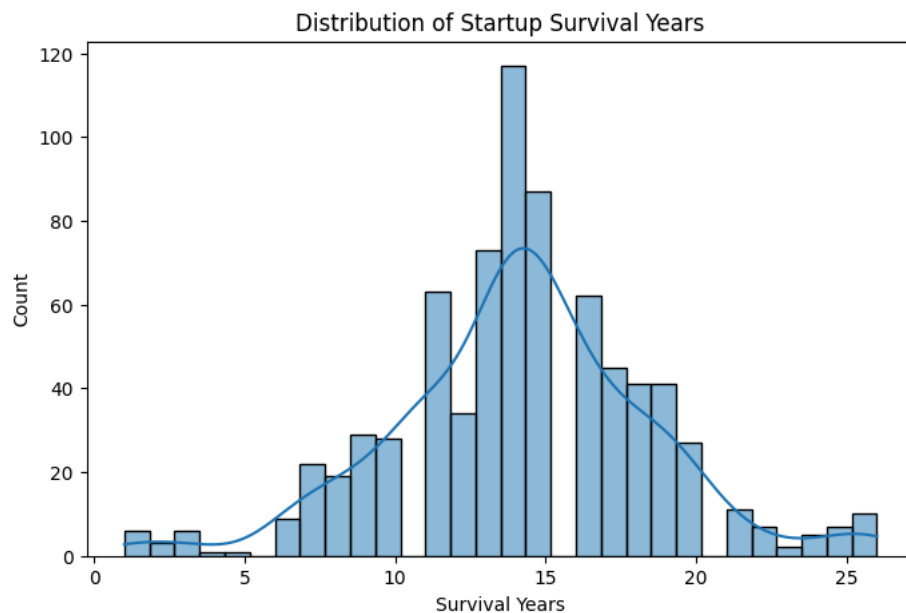
```
print(df[['Founded_Year', 'Funding_Rounds', 'Funding_Total_USD', 'Survival_Years']].describe())
```

	Founded_Year	Funding_Rounds	Funding_Total_USD	Survival_Years
count	756.000000	756.000000	7.560000e+02	756.000000
mean	2011.332011	2.408730	3.571378e+07	14.19709
std	3.953650	2.467458	1.710492e+08	4.29687
min	2000.000000	0.000000	0.000000e+00	1.00000
25%	2009.000000	1.000000	5.000000e+04	12.00000
50%	2012.000000	1.000000	2.000000e+06	14.00000
75%	2014.000000	3.000000	1.557809e+07	17.00000
max	2020.000000	23.000000	2.428380e+09	26.00000

```
# Survival years distribution
plt.figure(figsize=(8,5))

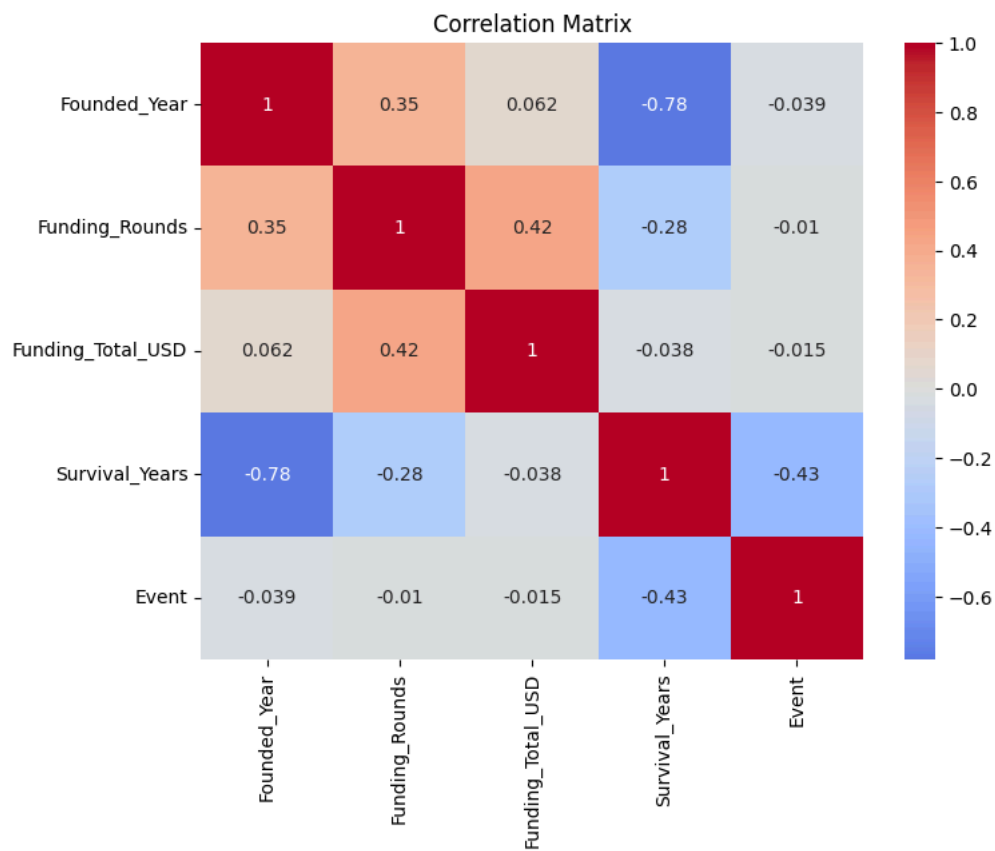
sns.histplot(data=df, x='Survival_Years', bins=30, kde=True)

plt.title('Distribution of Startup Survival Years')
plt.xlabel('Survival Years')
plt.show()
```



```
# Correlation heatmap
numeric_cols = ['Founded_Year', 'Funding_Rounds', 'Funding_Total_USD', 'Survival_Years', 'Event']
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Drop rows with NaN values for correlation calculation
df_clean = df[numeric_cols].dropna()
plt.figure(figsize=(8, 6))
sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix')
plt.show()
```



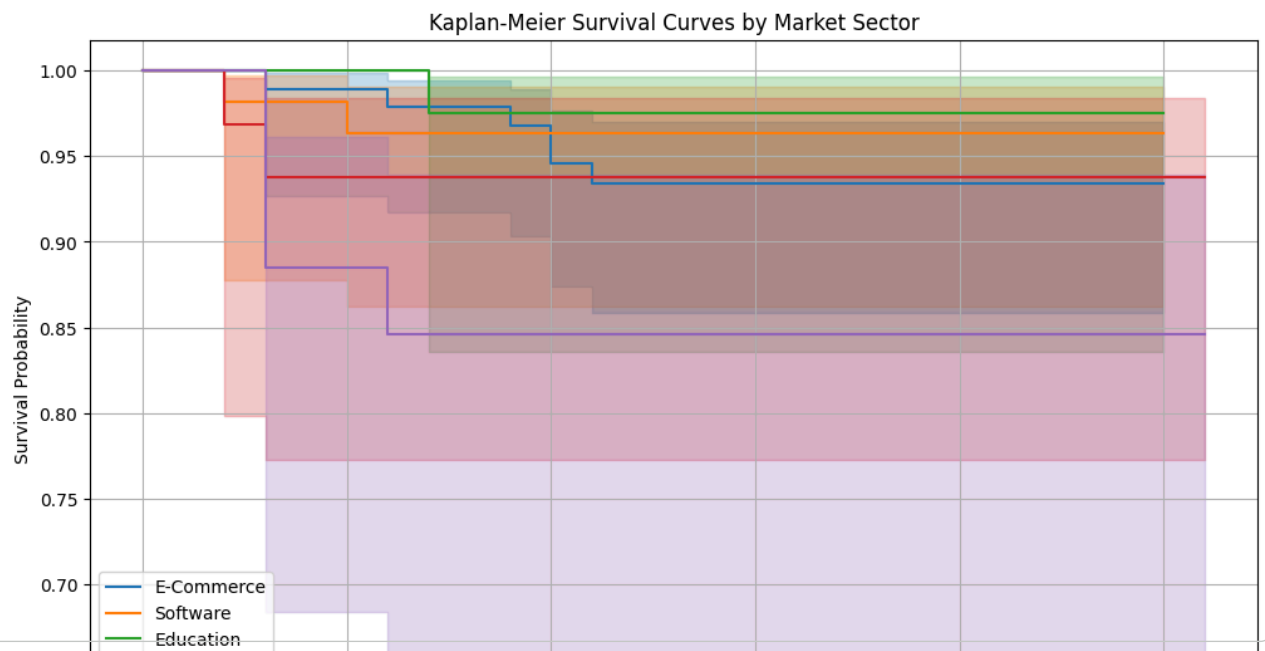
```
!pip install lifelines
```

```
Collecting lifelines
  Downloading lifelines-0.30.3-py3-none-any.whl.metadata (3.5 kB)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from lifelines) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from lifelines) (1.16.3)
Requirement already satisfied: pandas<3.0,>=2.1 in /usr/local/lib/python3.12/dist-packages (from lifelines) (2.2.2)
Requirement already satisfied: matplotlib>=3.0 in /usr/local/lib/python3.12/dist-packages (from lifelines) (3.10.0)
```



```
mask = df['Market'] == sector
kmf.fit(df[mask]['Survival_Years'], event_observed=df[mask]['Event'], label=sector)
kmf.plot()
```

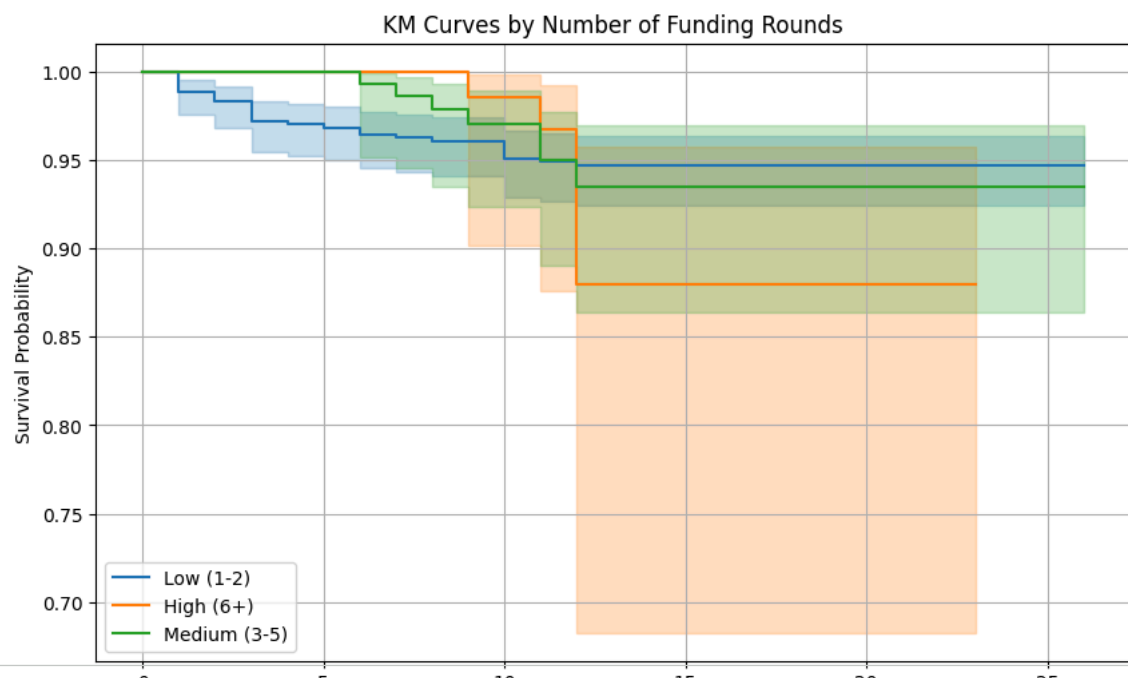
```
plt.title('Kaplan-Meier Survival Curves by Market Sector')
plt.xlabel('Years Since Founding')
plt.ylabel('Survival Probability')
plt.legend()
plt.grid(True)
plt.show()
```



```
df['Funding_Category'] = pd.cut(df['Funding_Rounds'], bins=[0, 2, 5, 100], labels=['Low (1-2)', 'Medium (3-5)', 'High (6+)'])
```

```
plt.figure(figsize=(10, 6))
for cat in df['Funding_Category'].dropna().unique():
    mask = df['Funding_Category'] == cat
    kmf.fit(df[mask]['Survival_Years'], event_observed=df[mask]['Event'], label=cat)
    kmf.plot()
```

```
plt.title('KM Curves by Number of Funding Rounds')
plt.xlabel('Years')
plt.ylabel('Survival Probability')
plt.legend()
plt.grid(True)
plt.show()
```



```
# TOP 5 CITIES BY NUMBER OF STARTUPS
```

```
top5_cities = df['City'].value_counts().head(5).index.tolist()
```

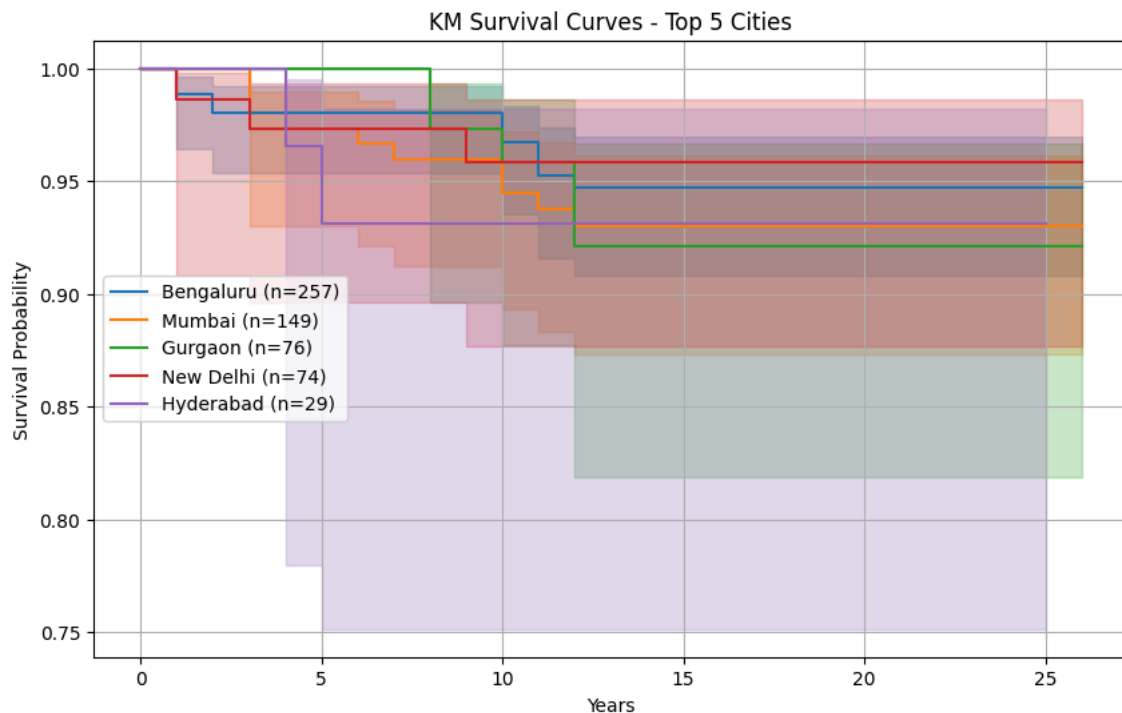
```

plt.figure(figsize=(10, 6))
kmf = KaplanMeierFitter()

for city in top5_cities:
    mask = df['City'] == city
    kmf.fit(df[mask]['Survival_Years'], df[mask]['Event'], label=f"{city} (n={mask.sum()})")
    kmf.plot()

plt.title('KM Survival Curves - Top 5 Cities')
plt.xlabel('Years')
plt.ylabel('Survival Probability')
plt.legend()
plt.grid(True)
plt.show()

```



```

# Create funding categories
df['Funding_Category'] = pd.qcut(
    df['Funding_Total_USD'],
    q=3,
    labels=['Low', 'Medium', 'High']
)

# Low vs High
from lifelines.statistics import multivariate_logrank_test

result = multivariate_logrank_test(
    df['Survival_Years'],
    df['Funding_Category'],
    df['Event']
)

print("\n1. FUNDING (ALL GROUPS):")
print(result)

```

```

1. FUNDING (ALL GROUPS):
<lifelines.StatisticalResult: multivariate_logrank_test>
      t_0 = -1
null_distribution = chi squared
degrees_of_freedom = 2
      test_name = multivariate_logrank_test

---
test_statistic    p  -log2(p)
          5.17  0.08    3.73

```

```

from lifelines.statistics import multivariate_logrank_test

from lifelines.statistics import multivariate_logrank_test

# Optional: drop missing values (recommended)

```



```
df_market_all = df[['Survival_Years', 'Event', 'Market']].dropna()

result_market_all = multivariate_logrank_test(
    df_market_all['Survival_Years'],
    df_market_all['Market'],
    df_market_all['Event']
)

print("\n1. MARKET (ALL GROUPS):")
print(result_market_all)
```

```
1. MARKET (ALL GROUPS):
<lifelines.StatisticalResult: multivariate_logrank_test>
      t_0 = -1
null_distribution = chi squared
degrees_of_freedom = 190
      test_name = multivariate_logrank_test

---
test_statistic      p  -log2(p)
      360.88 <0.005      39.83
```

```
# Optional: drop missing values (recommended)
df_city_all = df[['Survival_Years', 'Event', 'City']].dropna()

result_city_all = multivariate_logrank_test(
    df_city_all['Survival_Years'],
    df_city_all['City'],
    df_city_all['Event']
)

print("\n2. CITY (ALL GROUPS):")
print(result_city_all)
```

```
2. CITY (ALL GROUPS):
<lifelines.StatisticalResult: multivariate_logrank_test>
      t_0 = -1
null_distribution = chi squared
degrees_of_freedom = 53
      test_name = multivariate_logrank_test

---
test_statistic      p  -log2(p)
      136.91 <0.005      28.65
```

```
import pandas as pd
import numpy as np
from lifelines import CoxPHFitter

# Select columns
cox_df = df[['Market', 'City', 'Funding_Rounds', 'Funding_Total_USD',
            'Founded_Year', 'Survival_Years', 'Event']].dropna()

# Keep top 10 categories
top_markets = cox_df['Market'].value_counts().nlargest(10).index
top_cities = cox_df['City'].value_counts().nlargest(10).index

cox_df = cox_df[
    (cox_df['Market'].isin(top_markets)) &
    (cox_df['City'].isin(top_cities))
]

# Encode categorical variables
cox_df = pd.get_dummies(cox_df, columns=['Market', 'City'], drop_first=True)

# Transform features
cox_df['log_Funding'] = np.log1p(cox_df['Funding_Total_USD'])
cox_df['log_Rounds'] = np.log1p(cox_df['Funding_Rounds'])

# Feature list
features = ['log_Rounds', 'log_Funding', 'Founded_Year'] + \
    [c for c in cox_df.columns if c.startswith('Market_') or c.startswith('City_')]
```

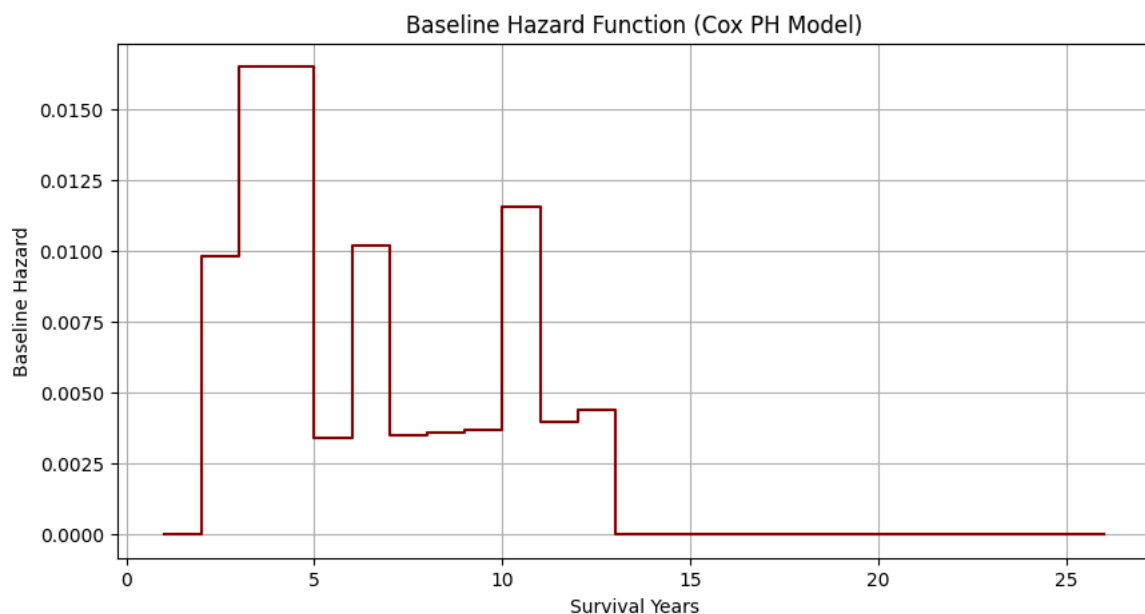
```
# Fit Cox model
cph = CoxPHFitter(penalizer=0.2) # slightly higher for stability
cph.fit(cox_df[features + ['Survival_Years', 'Event']],
        duration_col='Survival_Years',
        event_col='Event')
```

```
# Output
print(cph.summary[['coef', 'exp(coef)', 'p']].round(4))
```

	coef	exp(coef)	p
covariate			
log_Rounds	-0.0240	0.9763	0.9132
log_Funding	0.0092	1.0092	0.6102
Founded_Year	-0.0240	0.9763	0.3774
Market_E-Commerce	-0.0061	0.9939	0.9806
Market_Education	-0.2001	0.8186	0.5649
Market_FinTech	-0.0043	0.9957	0.9929
Market_Finance	0.3532	1.4237	0.3751
Market_Health Care	0.3770	1.4579	0.3937
Market_Mobile	-0.0256	0.9747	0.9464
Market_Real Estate	-0.0394	0.9614	0.9381
Market_Social Media	0.0380	1.0387	0.9447
Market_Software	-0.1224	0.8848	0.7008
City_Bengaluru	-0.1037	0.9015	0.6568
City_Chennai	0.4533	1.5735	0.3532
City_Gurgaon	-0.0255	0.9749	0.9455
City_Haryana	-0.2784	0.7570	0.8085
City_Hyderabad	0.0988	1.1039	0.8715
City_Mumbai	-0.0301	0.9703	0.9088
City_New Delhi	0.1300	1.1388	0.7247
City_Noida	0.0312	1.0317	0.9533
City_Pune	0.2503	1.2844	0.7239

```
# Get baseline cumulative hazard
baseline_cum_hazard = cph.baseline_cumulative_hazard_
baseline_hazard = baseline_cum_hazard.diff().fillna(0)

plt.figure(figsize=(10, 5))
plt.step(baseline_hazard.index, baseline_hazard.values, where='post', color='darkred')
plt.title('Baseline Hazard Function (Cox PH Model)')
plt.xlabel('Survival Years')
plt.ylabel('Baseline Hazard')
plt.grid(True)
plt.show()
```



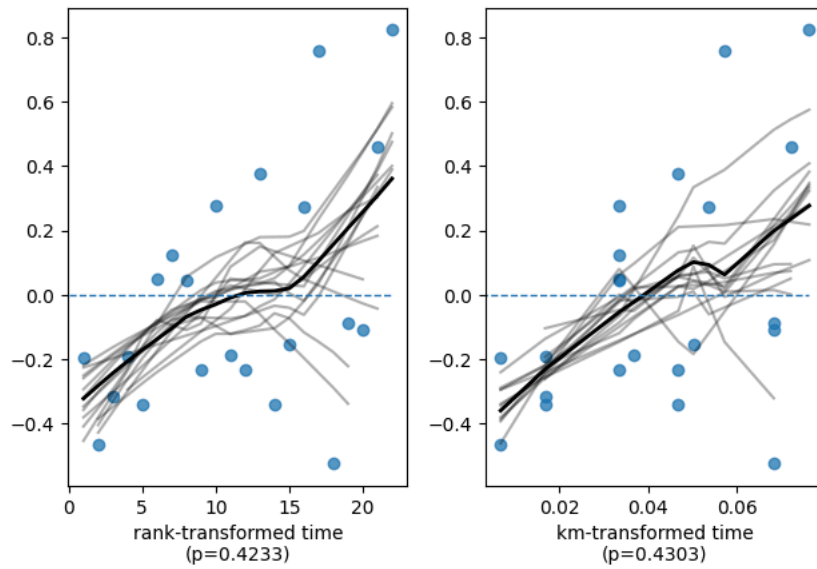
```
# Check proportional hazards assumption

import contextlib
import io

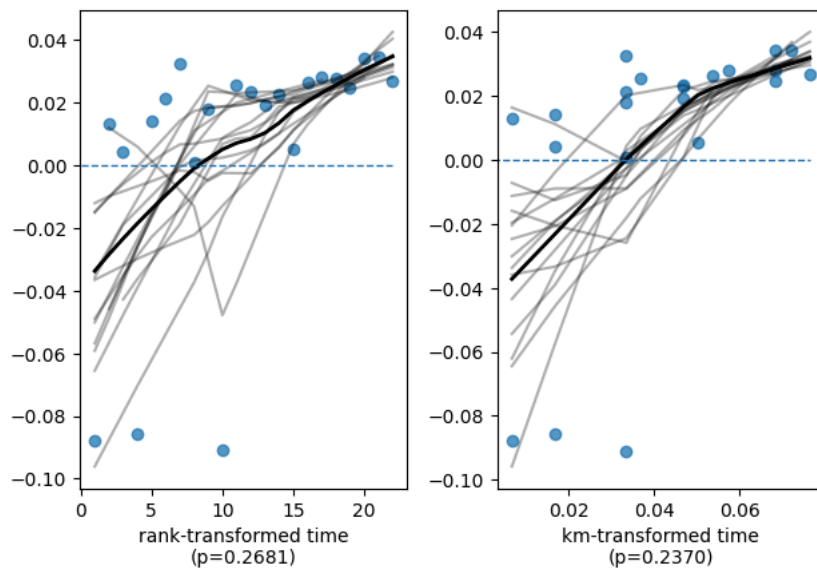
# Suppress text output but keep plots
with contextlib.redirect_stdout(io.StringIO()):
    cph.check_assumptions(
        cox_df[features + ['Survival_Years', 'Event']],
        p_value_threshold=0.05,
        show_plots=True # keep graphs
    )
```


/usr/local/lib/python3.12/dist-packages/lifelines/fitters/mixins.py:123: RuntimeWarning: More than 20 figures have been opened
fig = plt.figure()

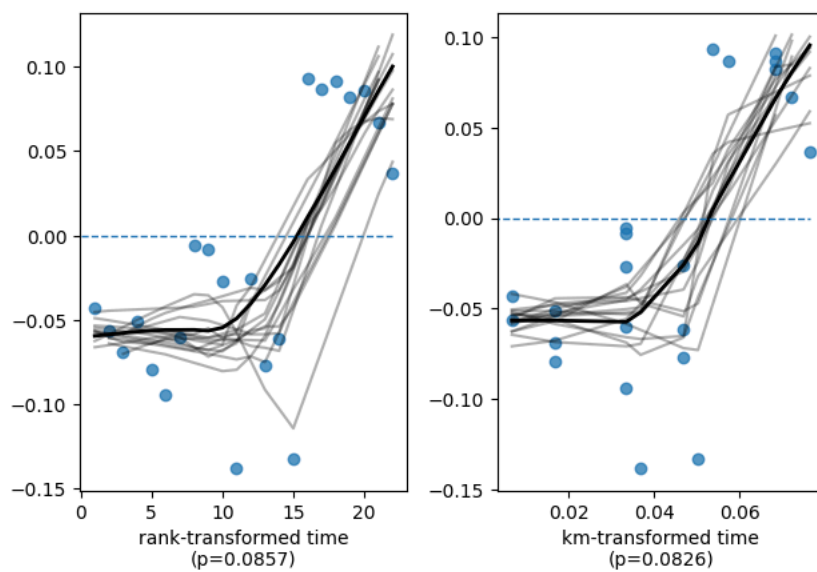
Scaled Schoenfeld residuals of 'log_Rounds'



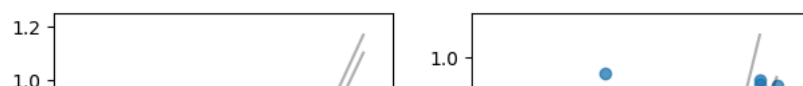
Scaled Schoenfeld residuals of 'log_Funding'

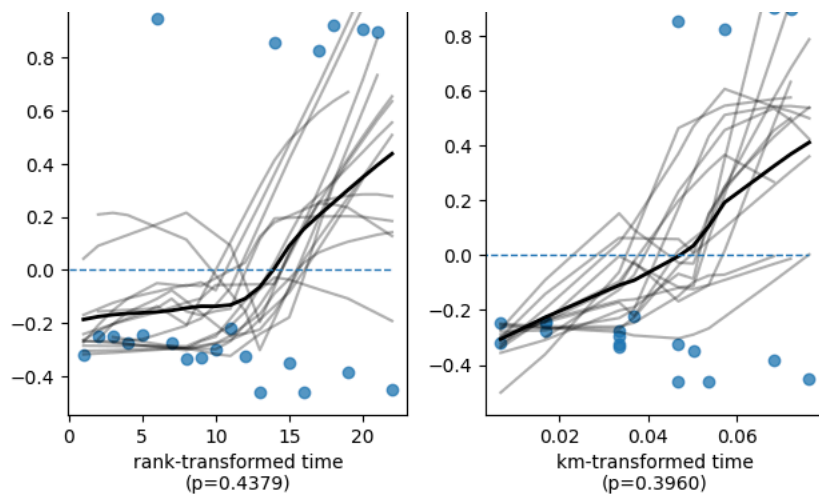


Scaled Schoenfeld residuals of 'Founded_Year'

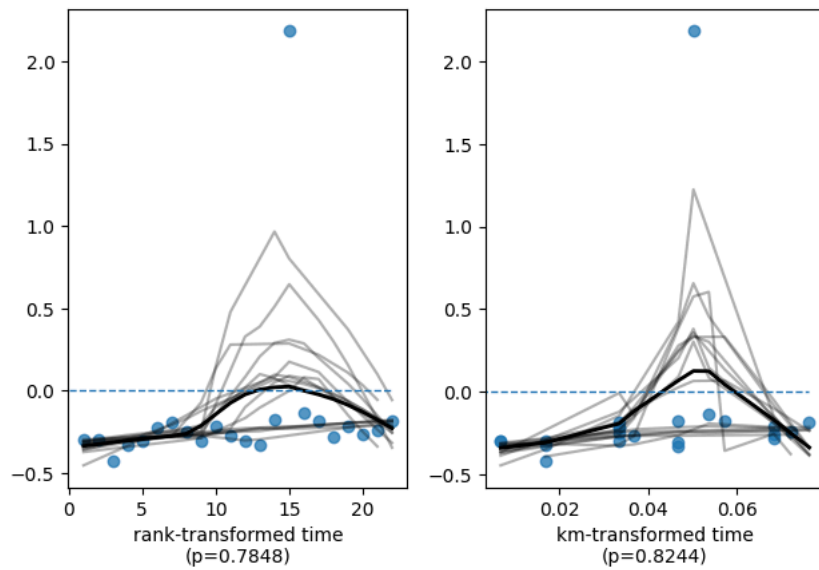


Scaled Schoenfeld residuals of 'Market_E-Commerce'

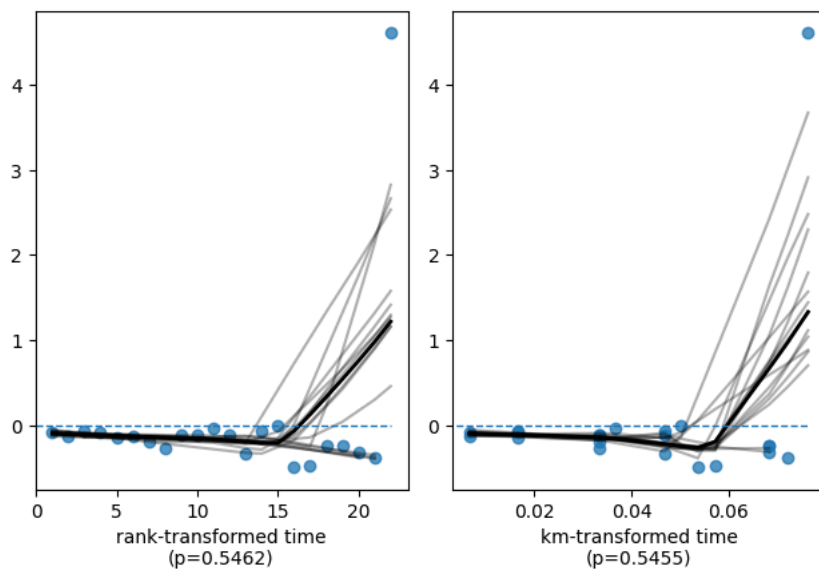




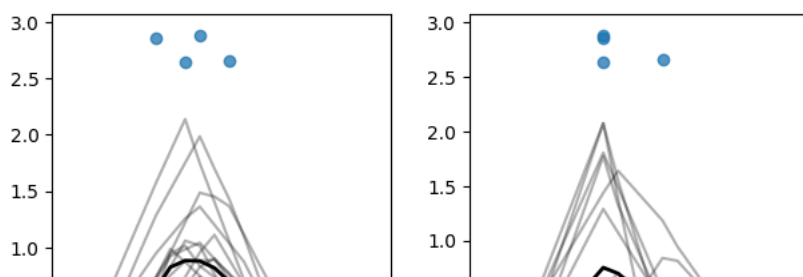
Scaled Schoenfeld residuals of 'Market_Education'

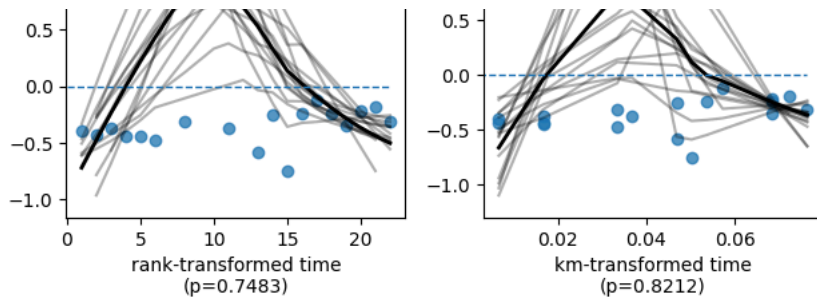


Scaled Schoenfeld residuals of 'Market_FinTech'

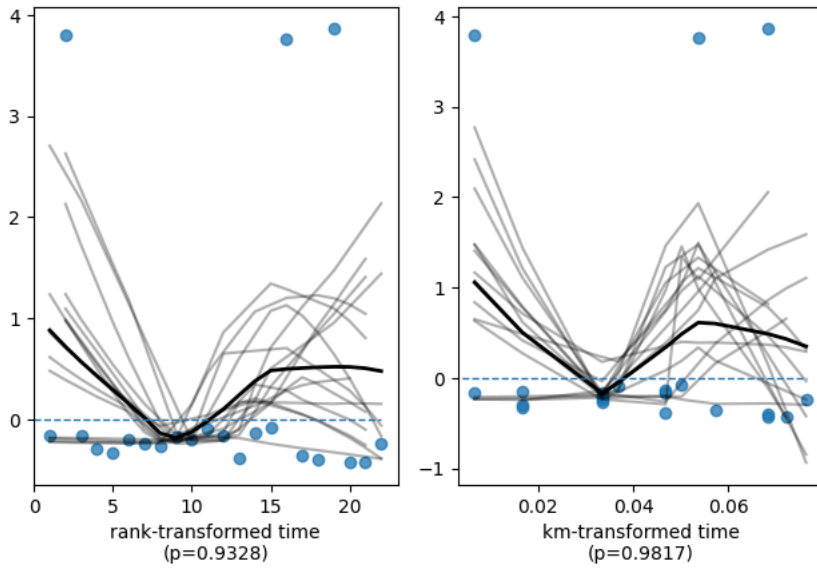


Scaled Schoenfeld residuals of 'Market_Finance'

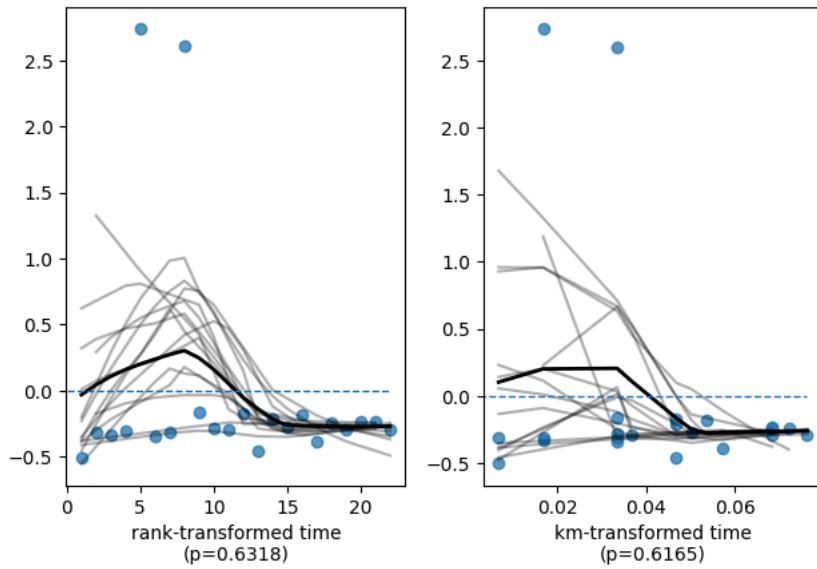




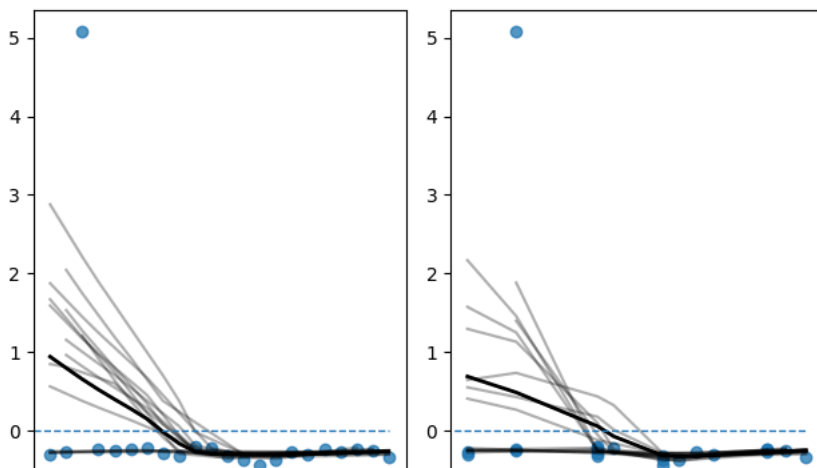
Scaled Schoenfeld residuals of 'Market_Health Care'

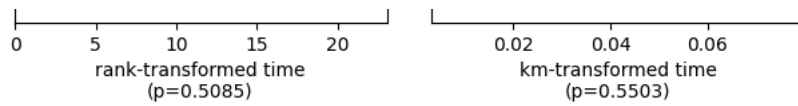


Scaled Schoenfeld residuals of 'Market_Mobile'

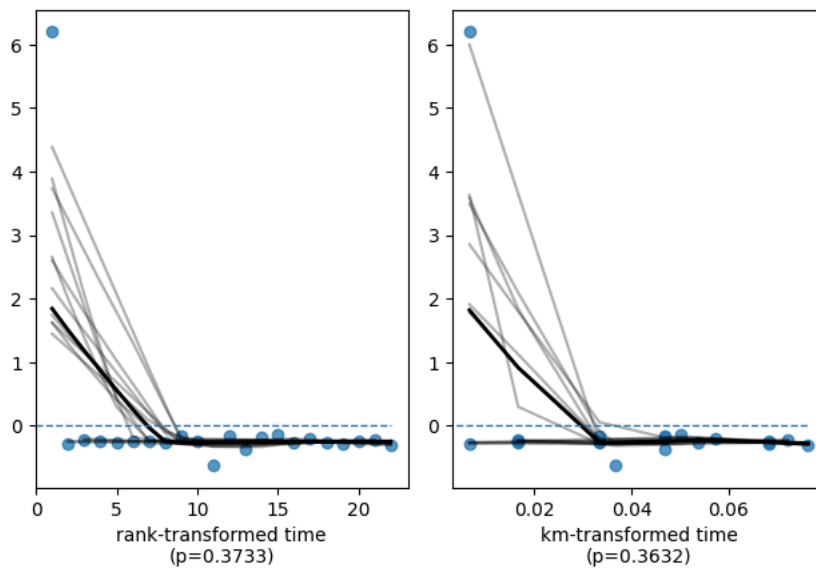


Scaled Schoenfeld residuals of 'Market_Real Estate'

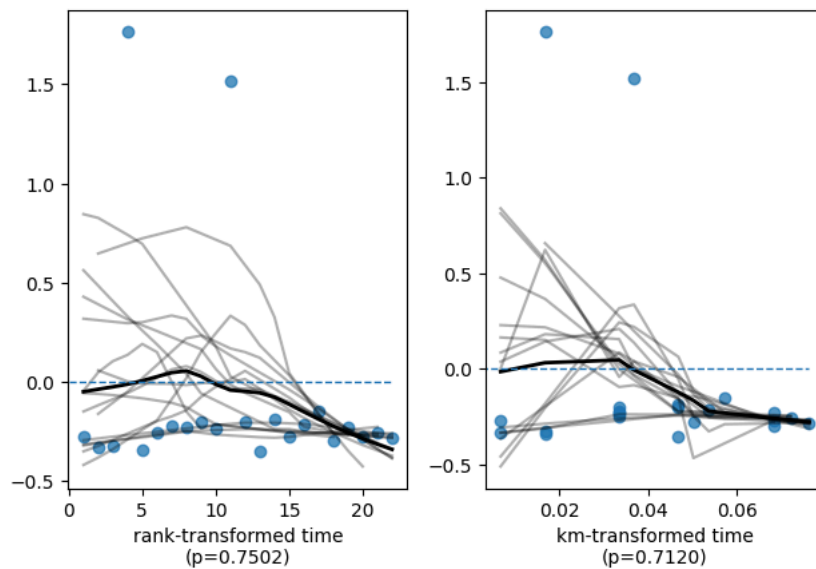




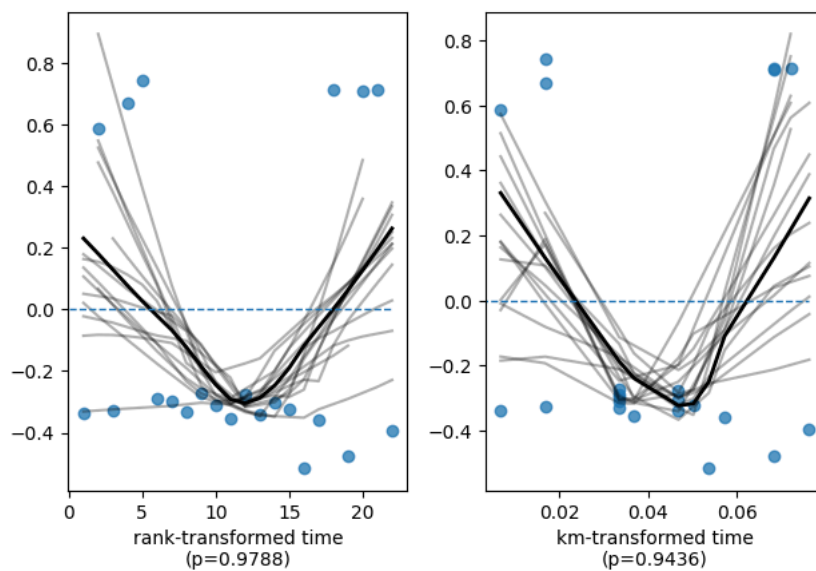
Scaled Schoenfeld residuals of 'Market_Social Media'



Scaled Schoenfeld residuals of 'Market_Software'



Scaled Schoenfeld residuals of 'City_Bengaluru'



Scaled Schoenfeld residuals of 'City_Chennai'

